GARAM TIA — RÉSUMÉ TOTAL

Architecture, outils, sécurité, données, tests et stratégie de développement
Document à revoir ensemble le soir pour arrêter la meilleure stratégie.

1. Objectif

Construire GARAM TIA comme une véritable application de jeu multijoueur, robuste, sécurisée, maintenable, testable, évolutive et pensée comme une plateforme SaaS multi-tenant. Aucun système ne garantit absolument zéro bug ; l'objectif est de réduire fortement les erreurs, les détecter vite et pouvoir récupérer après incident.

2. Principe fondamental : Godot n'est pas la base de données

Godot gère principalement l'état vivant de la partie : cartes en main, deck, joueur actif, tour, cartes jouées, animations, positions visuelles, drag & drop, interface et état courant de GameSession.

Le backend et la base de données gèrent ce qui doit survivre à la fermeture de l'application, une déconnexion ou un changement d'appareil : comptes, profils, soldes, transactions, parties, résultats, historique et autres données persistantes.

3. Architecture cible

Couche	Rôle
Godot	Client du jeu : scènes, interface, cartes, animations, interactions, état visuel et communication.
Backend	Autorité sur les règles sensibles, parties, joueurs, chronomètre, résultats, sécurité et logique métier.
Base de données	Comptes, profils, soldes, transactions, historique, résultats et paramètres persistants.
Authentification / droits	Identité des utilisateurs et permissions par rôle.
Git / GitHub	Versions, historique, sauvegarde distante et sécurité du développement.
Logs / monitoring	Surveillance, diagnostic et audit.

4. Multi-tenant SaaS

La même infrastructure doit servir de nombreux utilisateurs avec une isolation stricte des données et des autorisations. Un joueur ne doit jamais pouvoir consulter ou modifier les données d'un autre joueur en manipulant le client Godot.

5. Serveur autoritaire et anti-triche

Le serveur doit valider les cartes, tours, résultats, gains, commissions et opérations financières. Le chronomètre réel doit être contrôlé côté serveur ; Godot en affiche le compte à rebours.

6. Finances

Prévoir identifiant unique de transaction, prévention des doublons, validation serveur, journalisation, gestion des échecs et audit. Les opérations doivent éviter tout débit sans enregistrement correspondant ou toute double opération.

7. Reconnexion

Prévoir coupures Internet, fermeture forcée, changement de réseau, redémarrage et changement d'appareil. Le serveur doit connaître l'état valide de la partie et permettre une reprise cohérente lorsque les règles le prévoient.

8. Sécurité

Authentification, autorisation par rôle, protection des secrets et séparation des responsabilités. Les clés et secrets sensibles ne doivent pas être intégrés au client ni exposés sur GitHub.

9. Git et GitHub

Git conserve l'historique des versions ; les commits représentent des étapes stables. GitHub peut conserver le dépôt à distance. Avant les grosses modifications, on sécurise une version stable pour pouvoir revenir en arrière.

10. VS Code

VS Code peut devenir plus confortable lorsque le projet contient beaucoup de scripts. Il ne remplace pas Godot. Décision actuelle : rester avec l'éditeur intégré de Godot et introduire VS Code lorsque le volume de code le justifiera.

11. Codex

Codex peut aider à lire le projet, écrire ou modifier du code, travailler sur plusieurs fichiers et effectuer certaines tâches de développement. Il peut accélérer la logique globale, mais ses changements doivent être contrôlés, testés et protégés par Git.

12. Claude Code

Claude Code est une autre solution d'agent de programmation. Il sera comparé à Codex avant de choisir l'outil le plus adapté à notre projet et à notre méthode.

13. Rôle de ChatGPT

ChatGPT sert à réfléchir à l'architecture, au cahier des charges, aux règles, à la stratégie, aux instructions, à l'analyse des erreurs et à la validation des choix.

14. Qualité et tests

Prévoir progressivement des tests unitaires des règles, tests du système de cartes, tours, scores et chronomètre, tests backend/API, tests financiers, tests de reconnexion, tests de concurrence, tests d'intégration et tests de charge.

15. Logs et monitoring

Le système doit permettre de savoir ce qui s'est passé lors d'un incident : connexions, parties, actions importantes, résultats, transactions et erreurs. Le monitoring doit détecter pannes, ralentissements, erreurs et surcharge.

16. Environnements

Séparer au minimum développement → test → production. Les expérimentations ne doivent pas être faites directement sur les données réelles.

17. Sauvegardes et récupération

Prévoir des sauvegardes régulières et surtout tester la restauration. Il faut savoir récupérer le service après une panne ou une corruption de données.

18. Architecture du code

Éviter un seul gros script. Prévoir des modules séparés : UI, GameSession, CardSystem, PlayerSystem, NetworkSystem, authentification, portefeuille, transactions, communication backend, etc. Les règles métier doivent être centralisées et documentées.

19. CI/CD — plus tard

À maturité suffisante : automatiser tests, build, contrôles et déploiement afin de réduire les erreurs humaines.

20. Schéma global

ChatGPT → architecture et stratégie → **VS Code + Codex/Claude Code** → développement → **Git/GitHub** → versions → **Backend + Base de données** → données persistantes et autorité → **Godot** → client, interface, état vivant et expérience joueur.

21. Décisions du soir

1) Architecture exacte du backend et de la base de données.
2) Technologie backend/base de données.
3) Stratégie multi-tenant et isolation.
4) Mise en place de Git/GitHub.
5) Moment du passage à VS Code.
6) Choix Codex, Claude Code ou combinaison.
7) Organisation des branches et versions.
8) Méthode de tests.
9) Sécurité et anti-triche.

10) Déploiement, sauvegardes et récupération.
11) Ordre exact d'installation et d'utilisation des outils.

Principe directeur : ne pas ajouter des outils simplement parce qu'ils existent. Chaque outil doit avoir une fonction précise. L'objectif est de choisir une architecture solide dès le départ et de l'adopter progressivement.